

Day 56 -- Line-by-Line Syntax Explanation

day56_sequential_data_intro.py, segment by segment | MD Mutasim Billah

1. make_order_dataset() -- an Order-Sensitive Task, by Construction

sorting two positions to guarantee the label matches their order

```
p1, p2 = rng.choice(np.arange(pos_lo, pos_hi + 1), size=2, replace=False) # two distinct positions
p_high, p_low = (p1, p2) if p1 < p2 else (p2, p1) # sort so p_high is EARLIER in the sequence
if label == 0: # for label 0, swap which spike goes where
    p_high, p_low = p_low, p_high # low now comes first -- the 'wrong' order for label 1
```

This guarantees, by construction, that label 1 always means 'high spike's array index is smaller than low spike's index' and label 0 always means the opposite -- a clean, unambiguous order-detection task with no other signal a network could exploit as a shortcut.

2. PlainFCNet -- Position-Specific Weights, Made Explicit

the fixed-length check, and why W1's shape matters

```
self.W1 = rng.randn(T, hidden) * he_scale(T) # ONE ROW PER INPUT POSITION -- the crux of today's failure
def forward(self, X):
    if X.shape[1] != self.T: # NEW today: an explicit, honest fixed-length check
        raise ValueError(...) # rather than silently misbehaving on a shape mismatch
```

W1's shape (T, hidden) means row *t* is the ONLY set of weights that ever reads input position *t*. The explicit shape check turns an implicit numpy broadcasting error (which could be confusing) into a clear, intentional statement of this architecture's fixed-length limitation.

3. Conv1DNet -- Reusing 2D Conv Machinery for a 1D Signal

the reshape trick: a sequence becomes a 1-tall, 1-channel 'image'

```
self.Xr = X.reshape(m, 1, 1, self.T) # (m, T) sequence -> (m, 1 channel, 1 tall, T wide) 'image'
Zc = conv2d_forward_mc(self.Xr, self.Wc, self.bc, pad=0) # Day 53's already-verified 2D conv, unchanged
self.Zc = Zc.reshape(m, Zc.shape[1], Zc.shape[3]) # squeeze the trivial height=1 axis back out
```

conv2d_forward_mc doesn't know or care that height is 1 -- it just convolves over whatever height and width it's given. Setting height=1 and using a (C, 1, 1, k) filter makes the height dimension of the kernel also 1, so the only real convolving happens along width -- exactly a 1D convolution, with zero new conv math needed.

global_avg_pool_forward/backward -- today's one genuinely new backward-pass piece

```
def global_avg_pool_forward(A):  
    return A.mean(axis=2) # mean over the LENGTH axis -- one scalar per channel  
def global_avg_pool_backward(dOut, L):  
    return np.repeat(dOut[:, :, None], L, axis=2) / L # each of L positions gets an EQUAL 1/L  
share
```

The mean's own derivative: since every one of the L positions contributes equally (weight $1/L$) to the average, the gradient flowing back must also be split equally, $1/L$ to each position -- verified directly against a numeric gradient check before being trusted in the receptive-field experiment.